COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]
Extended by: Taylor Kearney

FIT3155: Advanced Algorithms and Data Structures

Week 6: Semi-numerical (or number-theoretic) algorithms

Faculty of Information Technology, Monash University

What is covered in this lecture?

This week's lecture deals with semi-numerical algorithms

- Algorithms involving (large) integers
  - Integer multiplication
  - Modular exponentiation
  - Miller-Rabin Primality Testing
    - This also introduces a new problem solving technique, **Randomized Algorithms**

Source material

- Cormen et al. Introduction to Algorithms (chapter 31)
- Knuth. The art of computer programming (Vol 2, chapter 4.3.3)
- Mathematicians discover the perfect way to multiply [click]

Introduction

- Number-theoretic algorithms are used widely today.
- Most prominent use is in cryptographic schemes.
- These schemes rely on **`large prime numbers`**.
- The **feasibility** of cryptographic schemes rely on the ability to generate large prime numbers.
- The **security** of cryptographic schemes rely on the **in**ability to efficiently factorize large numbers into their prime factors.

Computational complexity as a function of a size of input

- For the algorithms here, we need to change the way we think about the size of input.
- For instance, in sorting algorithms, we thought of the size of input in terms of the number of elements being sorted.
- Here, we will measure the input in terms of **number of bits** required to represent the input.
- For an integer n, the **number of bits** it takes to represent is $d = \lfloor \log n \rfloor + 1$ bits.
- So, a number-theoretic algorithm runs...:
  - ...in **linear** time, if it involves $O(d)$ computations.
  - ...in **quadratic** time if it involves $O(d^2)$ computations.
  - etc.
  - ...in **logarithmic** time if it involves $O(\log d)$ computations.

Integer multiplication

Long multiplication

The usual (manual) approach to multiply **large** numbers:

```
      123
    * 456
-------
      738      // = 123 *   6
+   6150       // = 123 *  50
+  49200       // = 123 * 400
-------
    56088
```

In general, to multiply two d digit numbers, the above long multiplication method can be converted into an algorithm that requires $O(d^2)$ single-digit multiplications (steps).

Can we do faster integer multiplication?

Brief history

- For millennia (until 1960s) we couldn't do better than in $O(d^2)$ steps.

Can we do faster integer multiplication?

Brief history

- For millennia (until 1960s) we couldn't do better than in $O(d^2)$ steps.
- In 1963 Anatoly Karatsuba gave a faster algorithm that requires $O(d^{\log_2 3})$ steps – this lecture will cover this algorithm.

Can we do faster integer multiplication?

Brief history

- For millennia (until 1960s) we couldn't do better than in $O(d^2)$ steps.
- In 1963 Anatoly Karatsuba gave a faster algorithm that requires $O(d^{\log_2 3})$ steps – this lecture will cover this algorithm.
- In 1971 Arnold Schönhage and Volker Strassen gave an even faster algorithm that takes $O(d \times \log d \times \log \log d)$ steps.

Can we do faster integer multiplication?

Brief history

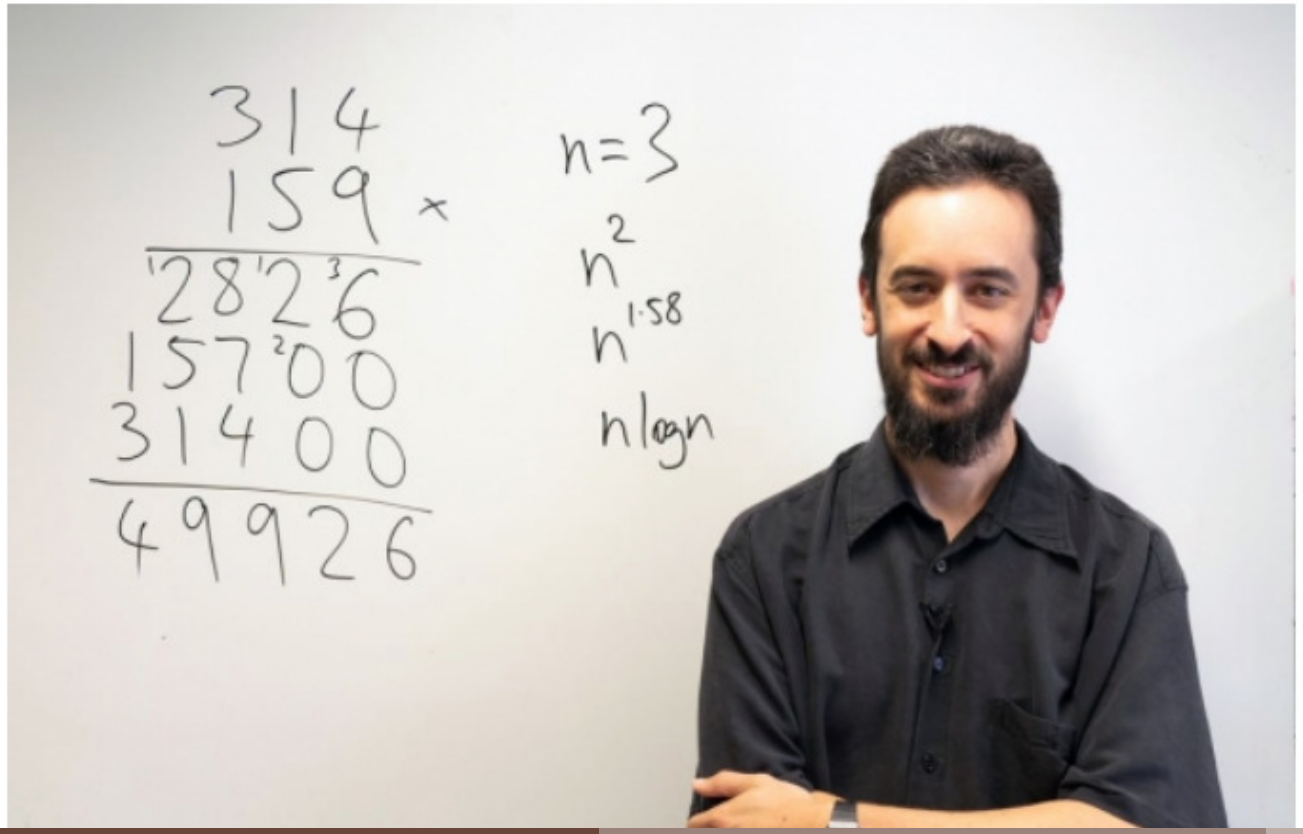
- For millennia (until 1960s) we couldn't do better than in $O(d^2)$ steps.
- In 1963 Anatoly Karatsuba gave a faster algorithm that requires $O(d^{\log_2 3})$ steps – this lecture will cover this algorithm.
- In 1971 Arnold Schönhage and Volker Strassen gave an even faster algorithm that takes $O(d \times \log d \times \log \log d)$ steps.
- **Quite recently**, in March 2019, David Harvey (Australian) and Joris Van Der Hoeven proposed an algorithm (the best possible one) for integer multiplication, requiring $O(d \times \log d)$ steps!

Can we do faster integer multiplication?

Maths whiz solves 48-year-old multiplication problem

04 APR 2019 | LACHLAN GILBERT

A UNSW Sydney mathematician has cracked a maths problem that has stood for almost half a century which will enable computers to multiply huge numbers together much more quickly.



Fast multiplication algorithm on (large) integers

$O(d^{\log_2 3})$-time integer multiplication

Divide and conquer approach

Assume we have two $2d$-bit numbers:

$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}$. That is, $u = 2^d U_1 + U_0$

$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}$. That is, $v = 2^d V_1 + V_0$

$O(d^{\log_2 3})$-time integer multiplication

Divide and conquer approach

Assume we have two $2d$-bit numbers:

$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}$. That is, $u = 2^d U_1 + U_0$

$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}$. That is, $v = 2^d V_1 + V_0$

Therefore, the product $uv = (2^d U_1 + U_0)(2^d V_1 + V_0)$ can be expanded as:

$O(d^{\log_2 3})$-time integer multiplication

Divide and conquer approach

Assume we have two $2d$-bit numbers:

$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}$. That is, $u = 2^d U_1 + U_0$

$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}$. That is, $v = 2^d V_1 + V_0$

Therefore, the product $uv = (2^d U_1 + U_0)(2^d V_1 + V_0)$ can be expanded as:

$$uv = 2^{2d}U_1V_1 + 2^d(U_0V_1 + U_1V_0) + U_0V_0$$

$O(d^{\log_2 3})$-time integer multiplication

Divide and conquer approach

Assume we have two $2d$-bit numbers:

$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}$. That is, $u = 2^d U_1 + U_0$

$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}$. That is, $v = 2^d V_1 + V_0$

Therefore, the product $uv = (2^d U_1 + U_0)(2^d V_1 + V_0)$ can be expanded as:

$$\begin{aligned} uv &= 2^{2d}U_1V_1 + 2^d(U_0V_1 + U_1V_0) + U_0V_0 \\ &= 2^{2d}U_1V_1 + 2^d(U_1V_1 - U_1V_1 + U_0V_1 + U_1V_0 - U_0V_0 + U_0V_0) + U_0V_0 \end{aligned}$$

$O(d^{\log_2 3})$-time integer multiplication

Divide and conquer approach

Assume we have two $2d$-bit numbers:

$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}$. That is, $u = 2^d U_1 + U_0$

$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}$. That is, $v = 2^d V_1 + V_0$

Therefore, the product $uv = (2^d U_1 + U_0)(2^d V_1 + V_0)$ can be expanded as:

$$\begin{aligned} uv &= 2^{2d}U_1V_1 + 2^d(U_0V_1 + U_1V_0) + U_0V_0 \\ &= 2^{2d}U_1V_1 + 2^d(U_1V_1 - U_1V_1 + U_0V_1 + U_1V_0 - U_0V_0 + U_0V_0) + U_0V_0 \\ &= 2^{2d}U_1V_1 + 2^d(U_1V_1 - (U_1 - U_0)(V_1 - V_0) + U_0V_0) + U_0V_0 \end{aligned}$$

$O(d^{\log_2 3})$-time integer multiplication

Divide and conquer approach

Assume we have two $2d$-bit numbers:

$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}$. That is, $u = 2^d U_1 + U_0$

$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}$. That is, $v = 2^d V_1 + V_0$

Therefore, the product $uv = (2^d U_1 + U_0)(2^d V_1 + V_0)$ can be expanded as:

$$\begin{aligned}
uv &= 2^{2d}U_1V_1 + 2^d(U_0V_1 + U_1V_0) + U_0V_0 \\
&= 2^{2d}U_1V_1 + 2^d(U_1V_1 - U_1V_1 + U_0V_1 + U_1V_0 - U_0V_0 + U_0V_0) + U_0V_0 \\
&= 2^{2d}U_1V_1 + 2^d(U_1V_1 - (U_1 - U_0)(V_1 - V_0) + U_0V_0) + U_0V_0 \\
&= (2^{2d} + 2^d)U_1V_1 - 2^d(U_1 - U_0)(V_1 - V_0) + (2^d + 1)U_0V_0
\end{aligned}$$

$O(d^{\log_2 3})$-time integer multiplication – continued

Divide and conquer approach

$$uv = (2^{2d} + 2^d)U_1V_1 - 2^d(U_1 - U_0)(V_1 - V_0) + (2^d + 1)U_0V_0$$

The form above decomposes the multiplication uv (problem of multiplying two $2d$-bit numbers) into:

- three d-bit multiplications:
  1. U_1V_1
  2. $(U_1 - U_0)(V_1 - V_0)$
  3. U_0V_0
- plus some simple left shifting*
- and some additions/subtractions.

*multiplication by any 2^k requires shifting the bits leftwards by k positions –reason!

$O(d^{\log_2 3})$-time integer multiplication – continued

Divide and conquer approach

$$uv = (2^{2d} + 2^d)U_1V_1 - 2^d(U_1 - U_0)(V_1 - V_0) + (2^d + 1)U_0V_0$$

The form above decomposes the multiplication uv (problem of multiplying two $2d$-bit numbers) into:

- three d-bit multiplications:
  1. U_1V_1
  2. $(U_1 - U_0)(V_1 - V_0)$
  3. U_0V_0
- plus some simple left shifting*
- and some additions/subtractions.

Time complexity

The time complexity of this divide-and-conquer approach can be expressed using the recurrence relationship: $T(2d) = 3T(d) + cd$, where c is some constant. Solving this recurrence yields the $O(d^{\log_2(3)})$ -time algorithm. Try solving this recurrence – this is your tute question.

*multiplication by any 2^k requires shifting the bits leftwards by k positions –reason!

Important: **Karatsuba's** recursive divide-and-conquer multiplication works in any base.

For integers u and v represented in any arbitrary base β, since $u = \beta^d U_1 + U_0$ and $v = \beta^d V_1 + V_0$:

Divide and conquer in base-β

$$uv = (\beta^{2d} + \beta^d) U_1 V_1 - \beta^d (U_1 - U_0)(V_1 - V_0) + (\beta^d + 1) U_0 V_0$$

$O(d^{\log_2 3})$-time integer multiplication – Self-study example

Consider two 4-bit numbers:

$u = 13_{\text{base-10}} = (\underbrace{11}_{U_1}\underbrace{01}_{U_0})_{\text{base-2}}$. That is, $u = 2^2(11) + 01$

$v = 14_{\text{base-10}} = (\underbrace{11}_{V_1}\underbrace{10}_{V_0})_{\text{base-2}}$. That is, $v = 2^2(11) + 10$

$O(d^{\log_2 3})$-time integer multiplication – Self-study example

Consider two 4-bit numbers:

$u = 13_{\text{base-10}} = (\underbrace{11}_{U_1}\underbrace{01}_{U_0})_{\text{base-2}}$. That is, $u = 2^2(11) + 01$

$v = 14_{\text{base-10}} = (\underbrace{11}_{V_1}\underbrace{10}_{V_0})_{\text{base-2}}$. That is, $v = 2^2(11) + 10$

Then:

$$\begin{aligned} uv &= (2^{2d} + 2^d)U_1V_1 - 2^d(U_1 - U_0)(V_1 - V_0) + (2^d + 1)U_0V_0 \\ &= (2^4 + 2^2)(11)(11) - 2^2(11 - 01)(11 - 10) + (2^2 + 1)(01)(10) \\ &= (2^4 + 2^2)(11)(11) - 2^2(10)(01) + (2^2 + 1)(01)(10) \end{aligned}$$

$O(d^{\log_2 3})$-time integer multiplication – Self-study example

Consider two 4-bit numbers:

$u = 13_{\text{base-10}} = (\underbrace{11}_{U_1}\underbrace{01}_{U_0})_{\text{base-2}}$. That is, $u = 2^2(11) + 01$

$v = 14_{\text{base-10}} = (\underbrace{11}_{V_1}\underbrace{10}_{V_0})_{\text{base-2}}$. That is, $v = 2^2(11) + 10$

Then:

$$\begin{aligned} uv &= (2^{2d} + 2^d)U_1V_1 - 2^d(U_1 - U_0)(V_1 - V_0) + (2^d + 1)U_0V_0 \\ &= (2^4 + 2^2)(11)(11) - 2^2(11 - 01)(11 - 10) + (2^2 + 1)(01)(10) \\ &= (2^4 + 2^2)(11)(11) - 2^2(10)(01) + (2^2 + 1)(01)(10) \end{aligned}$$

Now we compute $(11)(11)$, $(10)(01)$, and $(01)(10)$ using the same method...

$O(d^{\log_2 3})$-time integer multiplication – Self-study example

For $(11)(11)$, we have $2d = 2$ and $x = y = 2^1(1) + 1$,

$$\begin{aligned}
xy &= (2^{2d} + 2^d)X_1Y_1 - 2^d(X_1 - X_0)(Y_1 - Y_0) + (2^d + 1)X_0Y_0 \\
&= (2^2 + 2^1)(1)(1) - 2^1(1 - 1)(1 - 1) + (2^1 + 1)(1)(1) \\
&= (2^2 + 2^1)1 - 2^1(0)(0) + (2^1 + 1)(1) \\
&= 2^2 + 2^1 + 2^1 + 1 \\
&= 0100 + 0010 + 0010 + 0001 \\
&= 1001
\end{aligned}$$

The computations for $(10)(01) = 0010$, and $(01)(10) = 0010$ follow similarly.

$O(d^{\log_2 3})$-time integer multiplication – Self-study example

Finally, we obtain,

$$\begin{aligned} uv &= (2^4 + 2^2)1001 - 2^2(0010) + (2^2 + 1)(0010) \\ &= 10010000 + 100100 - 001000 + 001000 + 0010 \\ &= 10110110 \end{aligned}$$

Which is $13 \times 14 = 182$ in binary!

Divisibility and Divisors

The notion of one integer being **divisible** by another is key to theory of numbers.

Division theorem

For any integer a and any positive integer n, there exists unique integers q (quotient) and r (remainder) such that $a = qn + r$, where $0 \leq r < n$.

Quotient

The value $q = \lfloor \frac{a}{n} \rfloor$ is the **quotient** of the division.

Remainder

The value $a \bmod n = r$ is the **remainder** (sometimes called residue) of division.

Modular exponentiation (a^b mod n) on large integers

Modular Exponentiation

- A frequently occurring operation in number-theoretic computations is raising one number to a power, and then modulo dividing with another number.

$$a^b \text{ mod } n$$

- This is known as **modular exponentiation**.
- For really small integers, we can first compute a^b and then modulo divide the result with n to get an final answer.
- However, this method is not attractive even when dealing with not-so-large numbers, for example:

$$1729^{1023} \text{ mod } 75$$

- A popular algorithm for modular exponentiation is the method of **repeated-squaring**. – discussed below

Running example for modular exponentiation by repeated squaring: 7^{560} mod 561

We want to compute a^b mod n, where $a = 7, b = 560$ and $n = 561$ (i.e., compute 7^{560} mod 561)

Computation of 7^{560} mod 561

- Binary representation of $b = (560)_{10}$ is $(\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{0})_2$

Running example for modular exponentiation by repeated squaring: 7^{560} mod 561

We want to compute a^b mod n, where $a = 7, b = 560$ and $n = 561$ (i.e., compute 7^{560} mod 561)

Computation of 7^{560} mod 561

- Binary representation of $b = (560)_{10}$ is $(\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{0})_2$
- $a^b = 7^{560} = 7^{1\times2^9+0\times2^8+0\times2^7+0\times2^6+1\times2^5+1\times2^4+0\times2^3+0\times2^2+0\times2^1+0\times2^0}$

Running example for modular exponentiation by repeated squaring: 7^{560} mod 561

We want to compute a^b mod n, where $a = 7, b = 560$ and $n = 561$ (i.e., compute 7^{560} mod 561)

Computation of 7^{560} mod 561

- Binary representation of $b = (560)_{10}$ is $(\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{0})_2$
- $a^b = 7^{560} = 7^{1\times2^9+0\times2^8+0\times2^7+0\times2^6+1\times2^5+1\times2^4+0\times2^3+0\times2^2+0\times2^1+0\times2^0}$
- $\implies 7^{560} =$
$$\underbrace{(7^{2^9})^1(7^{2^8})^0(7^{2^7})^0(7^{2^6})^0(7^{2^5})^1(7^{2^4})^1(7^{2^3})^0(7^{2^2})^0(7^{2^1})^0(7^{2^0})^0}_{=7^{2^9}7^{2^5}7^{2^4}}$$

Running example for modular exponentiation by repeated squaring: 7^{560} mod 561

We want to compute a^b mod n, where $a = 7, b = 560$ and $n = 561$ (i.e., compute 7^{560} mod 561)

Computation of 7^{560} mod 561

- Binary representation of $b = (560)_{10}$ is $(\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{0})_2$
- $a^b = 7^{560} = 7^{1\times2^9+0\times2^8+0\times2^7+0\times2^6+1\times2^5+1\times2^4+0\times2^3+0\times2^2+0\times2^1+0\times2^0}$
- $\implies 7^{560} = \underbrace{(7^{2^9})^1(7^{2^8})^0(7^{2^7})^0(7^{2^6})^0(7^{2^5})^1(7^{2^4})^1(7^{2^3})^0(7^{2^2})^0(7^{2^1})^0(7^{2^0})^0}_{=7^{2^9}7^{2^5}7^{2^4}}$
- ...continued in the next slide.

...example continued: 7^{560} mod 561

Key property of modular arithmetic that we will use now is:

$x * y$ mod $z = (x$ mod $z * y$ mod $z)$ mod z

...example continued: 7^{560} mod 561

Key property of modular arithmetic that we will use now is:

$x * y$ mod $z = (x$ mod $z * y$ mod $z)$ mod z

- We saw in the previous slide that:
- $7^{560} = \underbrace{(7^{2^9})^{1}(7^{2^8})^{0}(7^{2^7})^{0}(7^{2^6})^{0}(7^{2^5})^{1}(7^{2^4})^{1}(7^{2^3})^{0}(7^{2^2})^{0}(7^{2^1})^{0}(7^{2^0})^{0}}_{=7^{2^9}7^{2^5}7^{2^4}}$

...example continued: 7^{560} mod 561

Key property of modular arithmetic that we will use now is:

$x * y$ mod $z = (x$ mod $z * y$ mod $z)$ mod z

- We saw in the previous slide that:
- $7^{560} = \underbrace{(7^{2^9})^{1}(7^{2^8})^{0}(7^{2^7})^{0}(7^{2^6})^{0}(7^{2^5})^{1}(7^{2^4})^{1}(7^{2^3})^{0}(7^{2^2})^{0}(7^{2^1})^{0}(7^{2^0})^{0}}_{=7^{2^9}7^{2^5}7^{2^4}}$
- To compute 7^{560} mod 561, using our modular arithmetic property we note that:

...example continued: 7^{560} mod 561

Key property of modular arithmetic that we will use now is:

$x * y$ mod $z = (x$ mod $z * y$ mod $z)$ mod z

- We saw in the previous slide that:
- $7^{560} = \underbrace{(7^{2^9})^1(7^{2^8})^0(7^{2^7})^0(7^{2^6})^0(7^{2^5})^1(7^{2^4})^1(7^{2^3})^0(7^{2^2})^0(7^{2^1})^0(7^{2^0})^0}_{=7^{2^9}7^{2^5}7^{2^4}}$
- To compute 7^{560} mod 561, using our modular arithmetic property we note that:

$$
\begin{aligned}
7^{560} \text{ mod } 561 &= 7^{2^9}7^{2^5}7^{2^4} \text{ mod } 561 \\
&= \left(7^{2^9} \text{ mod } 561 \times \left[7^{2^5}7^{2^4}\right] \text{ mod } 561\right) \text{ mod } 561 \\
&= \left(7^{2^9} \text{ mod } 561 \times \left\{\left[7^{2^5} \text{ mod } 561 \times 7^{2^4} \text{ mod } 561\right] \text{ mod } 561\right\}\right) \text{ mod } 561,
\end{aligned}
$$

and we can see that it is only the terms in our decomposition that correspond to there being a 1 in the binary representation of the exponent that are required.

...example continued: 7^{560} mod 561

To efficiently compute these terms we note that,

$$\begin{aligned} 7^{2^i} \text{ mod } 561 &= \left(7^{2^{i-1}} \times 7^{2^{i-1}}\right) \text{ mod } 561 \\ &= \left(\left[7^{2^{i-1}} \text{ mod } 561\right] \times \left[7^{2^{i-1}} \text{ mod } 561\right]\right) \text{ mod } 561, \end{aligned}$$

and so we can construct the required terms through *repeated squaring*.

...example continued: 7^{560} mod 561

To efficiently compute these terms we note that,

$$\begin{aligned} 7^{2^i} \text{ mod } 561 &= \left(7^{2^{i-1}} \times 7^{2^{i-1}}\right) \text{ mod } 561 \\ &= \left(\left[7^{2^{i-1}} \text{ mod } 561\right] \times \left[7^{2^{i-1}} \text{ mod } 561\right]\right) \text{ mod } 561, \end{aligned}$$

and so we can construct the required terms through *repeated squaring*.

- Start by computing 7^{2^0} mod 561 which is $= 7$.

...example continued: 7^{560} mod 561

To efficiently compute these terms we note that,

$$\begin{aligned} 7^{2^i} \text{ mod } 561 &= \left(7^{2^{i-1}} \times 7^{2^{i-1}}\right) \text{ mod } 561 \\ &= \left(\left[7^{2^{i-1}} \text{ mod } 561\right] \times \left[7^{2^{i-1}} \text{ mod } 561\right]\right) \text{ mod } 561, \end{aligned}$$

and so we can construct the required terms through *repeated squaring*.

- Start by computing 7^{2^0} mod 561 which is $= 7$.
- Since 7^{2^1} mod $561 = (7^{2^0} * 7^{2^0})$ mod 561

...example continued: 7^{560} mod 561

To efficiently compute these terms we note that,

$$\begin{aligned} 7^{2^i} \text{ mod } 561 &= \left(7^{2^{i-1}} \times 7^{2^{i-1}}\right) \text{ mod } 561 \\ &= \left(\left[7^{2^{i-1}} \text{ mod } 561\right] \times \left[7^{2^{i-1}} \text{ mod } 561\right]\right) \text{ mod } 561, \end{aligned}$$

and so we can construct the required terms through *repeated squaring*.

- Start by computing 7^{2^0} mod 561 which is $= 7$.
- Since 7^{2^1} mod $561 = (7^{2^0} * 7^{2^0})$ mod 561(applying the above key property) $= (\underbrace{7^{2^0} \text{ mod } 561}_{=7} * \underbrace{7^{2^0} \text{ mod } 561)}_{=7}$ mod $561 = 7^2$ mod $561 = 49$

...example continued: 7^{560} mod 561

To efficiently compute these terms we note that,

$$7^{2^i} \text{ mod } 561 = \left(7^{2^{i-1}} \times 7^{2^{i-1}}\right) \text{ mod } 561$$
$$= \left(\left[7^{2^{i-1}} \text{ mod } 561\right] \times \left[7^{2^{i-1}} \text{ mod } 561\right]\right) \text{ mod } 561,$$

and so we can construct the required terms through *repeated squaring*.

- Start by computing 7^{2^0} mod 561 which is $= 7$.
- Since 7^{2^1} mod $561 = (7^{2^0} * 7^{2^0})$ mod 561(applying the above key property) $= (\underbrace{7^{2^0} \text{ mod } 561}_{=7} * \underbrace{7^{2^0} \text{ mod } 561}_{=7})$ mod $561 = 7^2$ mod $561 = 49$
- Since 7^{2^2} mod $561 = (7^{2^1} * 7^{2^1})$ mod $561 =$ $(\underbrace{7^{2^1} \text{ mod } 561}_{=49} * \underbrace{7^{2^1} \text{ mod } 561}_{=49})$ mod $561 = 49^2$ mod $561 = 157$
- continue repeated squaring...

Full computation of 7^{560} mod 561 example

Since $7^{560} = 7^{2^9} 7^{2^5} 7^{2^4}$, the iterative (repeated-squaring) computation of the **result** of 7^{560} mod 561 is built on the values highlighted in red.

i	a^{2^i} mod n (repeated squaring)	**result**
0	7^{2^0} mod $561 = 7$	1
1	7^{2^1} mod $561 = (7 \times 7)$ mod $561 = 49$	1
2	7^{2^2} mod $561 = (49 \times 49)$ mod $561 = 157$	1
3	7^{2^3} mod $561 = (157 \times 157)$ mod $561 = 526$	1
4	7^{2^4} mod $561 = (526 \times 526)$ mod $561 = 103$	(1×103) mod $561 = 103$
5	7^{2^5} mod $561 = (103 \times 103)$ mod $561 = 511$	(103×511) mod $561 = 460$
6	7^{2^6} mod $561 = (511 \times 511)$ mod $561 = 256$	460
7	7^{2^7} mod $561 = (256 \times 256)$ mod $561 = 460$	460
8	7^{2^8} mod $561 = (460 \times 460)$ mod $561 = 103$	460
9	7^{2^9} mod $561 = (103 \times 103)$ mod $561 = 511$	(460×511) mod $561 = 1$
		Therefore, 7^{560} mod $561 = 1$

- Initialize **result**=1.
- Successively compute a^{2^i} mod n using repeated squaring starting from $a2^0$ mod n.
- If the ith bit of the exponent b is 1 (here $b = 560 = \overset{9}{1}\,\overset{8}{0}\,\overset{7}{0}\,\overset{6}{0}\,\overset{5}{1}\,\overset{4}{1}\,\overset{3}{0}\,\overset{2}{0}\,\overset{1}{0}\,\overset{0}{0}$ and $n = 561$), then update: **result** $=$ (**result** $* \, a^{2^i}$ mod n) mod n.
- Final **result** is the modular exponentiation of a^b mod n.

Miller-Rabin Randomized Primality testing

Prime Numbers

A prime number (or a prime) is a natural number **greater than 1** that has **NO** positive divisors/factors other than 1 and itself.

Primes in the first 100 natural numbers:

2	3	5	7	11
13	17	19	23	29
31	37	41	43	47
53	59	61	67	71
73	79	83	89	97

Primality Test

Question

Given a (arbitrarily long) number n as input, is n **prime**?

- **Prime numbers** have preoccupied human interest for time immemorial.
- Reliance on primes of modern crypto-systems (eg. RSA) makes **primality testing** an important algorithmic problem.
- From an algorithmic point-of-view, how efficiently can we check if a given number n is prime?

Before anything, we only care about primality testing when n is **odd**. If n is **even** (except $n = 2$), return 'Composite' directly. So, practically, we are testing primality of a **large odd number** n.

The distribution of prime numbers

- The prime number distribution function $\pi(n)$ specifies the number of primes that are less than or equal to n.
- For example:
  - $\pi(10) = 4$ (since primes ≤ 10 are 2,3,5, and 7).
  - $\pi(100) = 25$ (since there are 25 primes ≤ 100).
  - ...
  - $\pi(1,000,000,000) = 50,847,534$.

Asymptotic distribution of prime numbers

$$\lim_{n\to\infty} \frac{\pi(n)}{n/\ln(n)} = 1.$$

That is, $n/\ln(n)$ is a good approximation to $\pi(n)$.

Intuitively, this gives a rough **probability estimate** of $1/\ln(n)$ that any chosen n is a prime.

Naïve Algorithm for Primality testing – trial division

```
function naive_test1( n ) {
  for (k in the range 2 and n-1) {
     if ( n mod k == 0) return "Composite!";
  }
  return "Prime!"
}
```

Naïve Algorithm for Primality testing – trial division

```
function naive_test1( n ) {
  for (k in the range 2 and n-1) {
     if ( n mod k == 0) return "Composite!";
  }
  return "Prime!"
}
```

Complexity

- $O(n)$ **number of divisions**.
- Basic division algorithms between two d-**bit** integers is O(d^2).
  - A (decimal) number n requires $d = \lfloor \log(n) \rfloor + 1$ bits to represent. Therefore each division is $O(d^2)$
- Therefore, total complexity is $O(nd^2)$-time using the naïve algorithm above.
- However, this can be reduced to $O(\sqrt{n}d^2)$-time – How?

Congruence class modulo n

When dividing integers by n, we can divide them into n **Congruence classes**, based on the remainder r $(0 \leq r < n)$ that each integer leaves.

Congruence class modulo n

When dividing integers by n, we can divide them into n **Congruence classes**, based on the remainder r $(0 \leq r < n)$ that each integer leaves.

Congruence class definition

Two integers a and b are in the same congruence class modulo n, **if and only if** $(a - b)$ is an **integer multiple** of n. This relationship is denoted as:

$$a \equiv b \pmod{n}$$

Another way to look at this: Two integers a and b are in the same congruence class modulo n, **if and only if** the remainder when a is divided by n is same as the remainder when b is divided by n.

Congruence class modulo n

When dividing integers by n, we can divide them into n **Congruence classes**, based on the remainder r $(0 \leq r < n)$ that each integer leaves.

Congruence class definition

Two integers a and b are in the same congruence class modulo n, **if and only if** $(a - b)$ is an **integer multiple** of n. This relationship is denoted as:

$$a \equiv b \pmod{n}$$

Another way to look at this: Two integers a and b are in the same congruence class modulo n, **if and only if** the remainder when a is divided by n is same as the remainder when b is divided by n.

Example: A congruence class modulo $n = 7$

Any pair of integers $(a, b) \in \{\cdots, -11, -4, 3, 10, 17, \cdots\}$ are in the same congruence class modulo 7. That is, all numbers in that set leave a remainder of 3 when divided by 7.

Fermat's little theorem (FLT)

If p is a **prime number**, then for any integer a, the number $a^p - a$ is an **integer multiple** of p.
In the language/notation of **modular arithmetic**, this is expressed as

$$a^p \equiv a \pmod{p}.$$

If $a \nmid p$ (i.e. a is not divisible by p), Fermat's little theorem stated above is **equivalent** to the statement that $a^{p-1} - 1$ is an integer multiple of p:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Fermat's primality test for any integer n

This gives a **necessary** but NOT sufficient test of primality of any number n. If $a^{n-1} \not\equiv 1 \pmod{n}$, then n is definitely composite.
Otherwise, it is probably prime.

Randomized Fermat's little theorem (FLT) test

```
function FermatRandomizedTest( n ){
  a = choose random number in the range 1 < a < n-1;
  if (a^{n-1} is NOT CONGRUENT to 1 modulo n)
     return "Composite!"
  return "PROBABLY␣Prime!"
}
```

- As we said in the previous slide, this test is necessary but not sufficient.
- There exist **composite numbers** known as pseudoprimes (e.g., Carmichael numbers) that satisfy FLT for all bases a that are **coprime** to them, which means they behave like primes under the FLT test. For specific bases a, these composite numbers **pass** the FLT primality test, thus making the Fermat's base a, a **liar** (or a 'non-witness').
  - Refer modular exponentiation example on slide 22, which was actually a FLT test for $n = 561 = 3 \times 11 \times 17$, with the Fermat's base $a = 7$. Clearly, $a = 7$ is a **Fermat's liar** for $n = 561$.
- The test is probabilistic in the sense that, for a composite number n, the chance a random base a fails to show n is composite (i.e., is a Fermat liar) determines the error rate. Multiple tests reduce this error exponentially.

Miller-Rabin method of using FLT

Miller-Rabin primality testing improves on Fermat's Little Theorem by:

1. Performing **multiple tests** with different values of a chosen **randomly** as Fermat witnesses.
2. Using additional checks during modular exponentiation

$$a^{n-1} \pmod{n}$$

that reduce the probability of falsely calling a composite number as probably prime (discussed on the next slide).

Miller-Rabin test employs two key observations

Before anything, note that testing primality of some n using Miller Rabin involves testing only when n is **odd**. Even numbers > 2 can be trivially identified (based on their least significant bit being 0) and returned as 'composite'.

Observation 1

If n is an **odd** number, then $n - 1$ is even and can always be represented as

$$n - 1 = 2^s.t,$$

where t is some odd number ≥ 1.

Miller-Rabin test employs two key observations

Before anything, note that testing primality of some n using Miller Rabin involves testing only when n is **odd**. Even numbers > 2 can be trivially identified (based on their least significant bit being 0) and returned as 'composite'.

Observation 1

If n is an **odd** number, then $n-1$ is even and can always be represented as

$$n - 1 = 2^s.t,$$

where t is some odd number ≥ 1.

Observation 2: "**square-root test**"

From observation 1, we saw that $n - 1 = 2^s.t$, where t is odd. Denote $x_0 = a^{2^0 \cdot t} \bmod n$ and, in general, $x_i = a^{2^i t} \bmod n$, for all $1 \leq i \leq s$. Then, for any value of $a \in (1, n-1)$ (both sides exclusive), if

$$x_i \equiv 1 \pmod{n} \text{ and } x_{i-1} \not\equiv \pm 1^* \pmod{n}$$

then, n **is composite** and a is a Fermat's **liar**.

---

*n.b. $x_{i-1} \not\equiv \pm 1 \pmod{n}$ implies x_{i-1} does not leave a remainder of either 1 or $n-1$ under division with n

Intuition for observation 2

- $n - 1 = 2^s.t$
- $\implies a^{n-1}$ mod $n = a^{2^s.t}$ mod n.
- We will compute $a^{2^s.t}$ mod n iteratively starting with...
- ... $a^{2^0.t}$ mod n (denoted as x_0).
- This result is progressively squared (modulo n) yielding:
  - ...$a^{2^1.t}$ mod n (denoted as x_1), which in turn yields
  - ...$a^{2^2.t}$ mod n (denoted as x_2), which in turn yields
  - ...$a^{2^3.t}$ mod n (denoted as x_3)...
  - ...and so on until
- This succesive squaring method yields a sequence of numbers: $\langle x_0, x_1, x_2, \cdots x_s \rangle$

Intuition for observation 2...continued

- The following **cases** apply to this sequence: $\langle x_0, x_1, x_2, \cdots x_s \rangle$
  1. $x_s \equiv r \not\equiv 1$. That is, the sequence does not end with $x_s \equiv 1$ result.
     - ★ Return 'Composite' (no further tests needed as this is conclusive)
  2. Some $x_{i-1} \equiv r \not\equiv -1$, and $x_i = 1$ is the first observed 1 in the sequence, that is, the sequence is of the form $\langle \cdots, r, 1, 1, 1 \cdots 1 \rangle$.
     - ★ Return 'Composite' (no further tests needed as this is conclusive)
  3. Some $x_{i-1} \equiv -1$, and $x_i \equiv 1$. That is, the sequence is of the form: $\langle \cdots, -1, 1, 1, 1 \cdots 1 \rangle$.
     - ★ 'Probably prime' (needs more FLT tests for more accuracy)
  4. $x_0 \equiv x_1 \equiv x_2 \equiv \cdots \equiv x_s \equiv 1$. That is, the sequence is **all** ones.
     - ★ 'Probably prime' (needs more FLT tests for more accuracy)

Miller-Rabin's Randomized Primality testing algorithm

Assumption: Identify 2 and 3 as primes, and all even numbers as composite, separately.

```
/* Input: Odd n>3 being tested for primality using MR.
   Input: k, number of tests  */
function MillerRabinRandomizedPrimality( n, k ) {
  /* Factor n-1 as 2^s.t, where t is odd */
  s = 0, t = n-1;
  while (t is even) {
    s = s+1; t = t/2
  } // at this stage, n-1 will be 2^s.t, where t is odd
  /*conduct k FLT tests*/
  loop (k times) {
    a = choose random number in [2...n-2];
    compute x_0        /*x_0 = a^t mod n*/
    if (x_0 == 1) continue                          /*case 4: slide 34 */
    loop (i in [1...s]) {
      compute x_i;     /*x_i = (x_{i-1})^2 mod n */
      if (x_i == 1) { /*x_i = a^{2^i}t mod n = 1*/
        if (x_{i-1} != n-1) return "Composite";    /*case 2: slide 34 */
        else {x_s = 1; break;}                     /*case 3: slide 34 */
      }
    }
    if (x_s  != 1 modulo n)  return "Composite!"; /*case 1: slide 34 */
  }
  return "probably_prime"; // accuracy depends on k
}
```

Accuracy of Miller-Rabin's algorithm

- The more the number of a's that are tested, the more is the accuracy of this test.
- There is a proof (NOT EXAMINABLE) that shows that this algorithm declares a composite number incorrectly prime with a probability of at most $\frac{1}{4^k}$, over k tests.
- That is, if $k = 64$, then the probability of a given odd composite number n to be incorrectly called a prime is $\frac{1}{2^{128}}$.

Next week

Fibonacci Heaps

-=o0o=-

END

-=o0o=-